

Project: Smart Agriculture System

Name: Sarvaiya Bhautik Mukeshbhai

Company: Skilldzire Technologies pvt. Ltd.

Internship Duration: 15 Days
(03/07/2026 To 17/07/2026)

Submission Date: 17/07/2026

INDEX

NO.	CHAPTER	Page No.
1.	Introduction	3
2.	Project Objectives	5
3.	Project Overview	6
4.	Hardware Components	8
5.	Software and Tools	11
6.	System Architecture	14
7.	Circuit Design	16
8.	Implementation Programming	18
9.	Testing and Results	24
10.	Challenges Faced	26
11.	Learning Outcomes	27
12.	Conclusion	28
13.	References	29

1. Introduction

1.1 Background of Smart Agriculture

Agriculture is one of the most important sectors for food production and economic development. Traditional farming methods often depend on manual observation and fixed irrigation schedules, which can lead to excessive water consumption, reduced crop productivity, and increased labor requirements. With the advancement of technology, agriculture is becoming smarter through the use of sensors, wireless communication, and cloud computing.

Smart agriculture is a modern farming approach that uses Internet of Things (IoT) technology to monitor environmental conditions such as temperature, humidity, and soil moisture in real time. By collecting and analyzing this data, farmers can make informed decisions about irrigation and crop management. This improves resource utilization, increases productivity, and supports sustainable farming practices.

1.2 Need for IoT in Agriculture

The Internet of Things (IoT) enables agricultural devices to communicate over the internet and provide real-time information from the field. Using IoT, farmers can monitor crop conditions remotely, automate irrigation systems, and receive timely updates about environmental changes.

The need for IoT in agriculture arises from several challenges faced in conventional farming, including water scarcity, unpredictable weather conditions, labor shortages, and inefficient resource management. IoT-based systems help address these issues by continuously monitoring field conditions and automating actions based on sensor data. This leads to improved crop health, optimized water usage, reduced operational costs, and increased farming efficiency.

1.3 Problem Statement

Traditional irrigation methods generally operate on fixed schedules or manual control without considering the actual condition of the soil or surrounding environment. This can result in over-irrigation, under-irrigation, water wastage, and reduced crop yield. Additionally, farmers often need to visit fields regularly to monitor environmental conditions, making the process time-consuming and labor-intensive.

To overcome these challenges, an IoT-based Smart Agriculture System has been developed using the ESP32 microcontroller. The system continuously monitors temperature, humidity, and soil moisture using sensors. The collected data is displayed locally on an OLED display and transmitted to the ThingsBoard cloud.

platform through the MQTT protocol. Based on the soil moisture level, the system automatically controls the irrigation relay, ensuring efficient water management and reducing manual intervention. This solution provides real-time monitoring, automation, and remote accessibility, making agricultural operations more efficient and sustainable.

2. Project Objectives

The primary objective of this project is to design and develop an **IoT-Based Smart Agriculture Monitoring and Irrigation System** that enables real-time monitoring of environmental conditions and automates irrigation using the ESP32 microcontroller and the ThingsBoard cloud platform.

The specific objectives of the project are:

1. To design and implement a smart agriculture system using the **ESP32 DevKit V1** microcontroller.
2. To monitor environmental parameters such as **temperature** and **humidity** using the **DHT22 sensor**.
3. To simulate soil moisture measurement using a **potentiometer** in the Wokwi simulation environment.
4. To display real-time sensor readings and pump status on an **OLED SSD1306 display**.
5. To automatically control the irrigation system by activating a **relay module** when the soil moisture level falls below a predefined threshold.
6. To establish wireless communication using the **ESP32's built-in Wi-Fi** module.
7. To transmit sensor data securely to the **ThingsBoard IoT cloud platform** using the **MQTT communication protocol**.
8. To visualize real-time agricultural data on a cloud dashboard for remote monitoring and analysis.
9. To reduce water wastage by implementing an automated irrigation control mechanism based on soil moisture conditions.
10. To demonstrate the practical application of **Embedded Systems, IoT, Cloud Computing**, and **Wireless Communication** technologies in modern agriculture.

3. Project Overview

3.1 Brief Description of the System

The **IoT-Based Smart Agriculture Monitoring and Irrigation System** is designed to monitor important environmental parameters and automate irrigation using Internet of Things (IoT) technology. The system is built around the **ESP32 DevKit V1** microcontroller, which serves as the central processing unit and provides built-in Wi-Fi connectivity for cloud communication.

The system continuously measures **temperature** and **humidity** using a **DHT22 sensor**, while **soil moisture** is simulated using a **potentiometer** in the Wokwi simulation environment. The collected sensor data is displayed in real time on an **OLED SSD1306 display** and simultaneously transmitted to the **ThingsBoard cloud platform** using the **MQTT communication protocol**.

Based on the soil moisture level, the ESP32 automatically controls a **relay module** that represents the irrigation water pump. When the soil moisture level falls below the predefined threshold, the relay is activated to simulate the irrigation process. Once sufficient moisture is detected, the relay is switched OFF automatically. This automation helps optimize water usage, reduce manual intervention, and improve agricultural efficiency.

The project was developed and tested using the **Wokwi online simulation platform**, enabling hardware and software validation without requiring physical components.

3.2 Features

The major features of the developed system are:

- Real-time monitoring of **temperature** and **humidity** using the DHT22 sensor.
- Soil moisture monitoring through a simulated analog sensor (potentiometer).
- Automatic irrigation control using a relay based on soil moisture levels.
- Real-time display of sensor values and pump status on an OLED display.
- Wireless communication using the ESP32's built-in Wi-Fi module.
- Cloud-based monitoring through the ThingsBoard IoT platform.
- Data transmission using the MQTT protocol for reliable communication.
- Visual indication of pump status using an LED.
- Continuous monitoring with automatic sensor updates.

- Scalable architecture suitable for future expansion with additional sensors and cloud-based analytics.

3.3 Technologies Used

Technology	Purpose
ESP32 DevKit V1	Main microcontroller with built-in Wi-Fi
Embedded C / Arduino Programming	Development of the application logic
Arduino IDE	Writing, compiling, and uploading the program
Wokwi Simulator	Online simulation and testing of the complete hardware circuit
ThingsBoard IoT Platform	Cloud-based monitoring, visualization, and data storage
MQTT Protocol	Lightweight communication protocol for sending telemetry data to the cloud
Wi-Fi Communication	Wireless internet connectivity between ESP32 and ThingsBoard
DHT22 Sensor	Measurement of temperature and humidity
OLED SSD1306 Display	Local display of sensor readings and pump status
Relay Module	Automatic control of the irrigation pump
Potentiometer	Simulation of soil moisture sensor values in Wokwi
LED Indicator	Visual indication of pump operation

Table 3.3.1:Technologies Used

4. Hardware Components

This project uses several hardware components to monitor environmental conditions, automate irrigation, and display real-time information. Each component performs a specific function within the Smart Agriculture System.

4.1 ESP32 DevKit V1

The **ESP32 DevKit V1** is the main controller of the system. It is a powerful microcontroller with built-in Wi-Fi and Bluetooth capabilities, making it suitable for Internet of Things (IoT) applications. In this project, the ESP32 collects data from sensors, processes the information, controls the relay module, displays sensor values on the OLED screen, and transmits telemetry data to the ThingsBoard cloud platform using the MQTT protocol.

Specifications:

- Dual-core 32-bit processor
- Operating Voltage: 3.3V
- Built-in Wi-Fi and Bluetooth
- Multiple GPIO, ADC, UART, SPI, and I²C interfaces
- Low power consumption

Role in the Project:

- Reads sensor data
- Controls irrigation relay
- Drives OLED display
- Connects to Wi-Fi
- Sends data to ThingsBoard

4.2 DHT22 Temperature and Humidity Sensor

The **DHT22** is a digital sensor used to measure ambient temperature and relative humidity with good accuracy. It communicates with the ESP32 through a single digital data pin.

Specifications:

- Operating Voltage: 3.3V–6V
- Temperature Range: -40°C to 80°C
- Humidity Range: 0% to 100% RH
- Digital Output

Role in the Project:

- Measures environmental temperature.
- Measures relative humidity.
- Sends real-time data to the ESP32 for processing and cloud monitoring.

4.3 Potentiometer (Soil Moisture Simulation)

In the Wokwi simulation environment, a **potentiometer** is used to simulate the output of a soil moisture sensor. Rotating the potentiometer changes the analog voltage, allowing different soil moisture conditions to be tested without physical hardware.

Specifications:

- Analog variable resistor
- Operating Voltage: 3.3V
- Analog Output

Role in the Project:

- Simulates soil moisture levels.
- Provides analog input to the ESP32.
- Controls automatic irrigation logic.

4.4 Relay Module

The **relay module** acts as an electronic switch that controls the irrigation pump. When the soil moisture level falls below the predefined threshold, the ESP32 activates the relay to simulate turning the water pump ON. When sufficient moisture is detected, the relay is turned OFF automatically.

Specifications:

- Operating Voltage: 5V
- Digital Control Input
- Supports switching external loads

Role in the Project:

- Controls the irrigation system.
- Provides automatic switching based on soil moisture.
- Simulates water pump operation.

4.5 OLED Display (SSD1306)

The **SSD1306 OLED Display** is used to display real-time information such as temperature, humidity, soil moisture percentage, and pump status. It communicates with the ESP32 using the I²C interface.

Specifications:

- Display Size: 0.96 inch
- Resolution: 128 × 64 pixels
- Communication Protocol: I²C
- Operating Voltage: 3.3V

Role in the Project:

- Displays sensor readings.
- Displays irrigation pump status.
- Provides local monitoring without requiring cloud access.

4.6 LED Indicator

An **LED** is used as a visual indicator of the irrigation pump status during simulation. Whenever the relay is activated, the LED also turns ON to indicate that irrigation is in progress.

Specifications:

- Operating Voltage: 2V–3.3V (with current-limiting resistor)
- Low power consumption

Role in the Project:

- Indicates pump operation.
- Provides quick visual feedback during testing.
- Helps verify relay operation in the simulation.

5. Software and Tools

The development and implementation of the **IoT-Based Smart Agriculture Monitoring and Irrigation System** required several software applications, simulation platforms, and communication technologies. These tools were used for programming, simulation, cloud connectivity, and real-time data monitoring. Each software tool played an important role in the successful development and testing of the project.

5.1 Arduino IDE

The **Arduino Integrated Development Environment (IDE)** was used to develop the firmware for the ESP32 microcontroller. It provides a user-friendly programming environment for writing, compiling, debugging, and uploading Embedded C/C++ code. The Arduino IDE also supports various libraries required for sensor interfacing, OLED display control, Wi-Fi communication, and MQTT connectivity.

Purpose

- Writing and editing the ESP32 program.
- Compiling and verifying the source code.
- Uploading the program to the ESP32.
- Managing external libraries.

Features

- Simple and easy-to-use interface.
- Supports ESP32 development boards.
- Built-in Serial Monitor for debugging.
- Large collection of open-source libraries.

5.2 Wokwi Simulator

Wokwi is an online electronics simulation platform used to design and test embedded systems without requiring physical hardware. In this project, Wokwi was used to simulate the complete Smart Agriculture System, including the ESP32, DHT22 sensor, potentiometer, relay module, OLED display, and LED. It allowed the project to be tested and verified before real hardware implementation.

Purpose

- Designing the virtual circuit.
- Simulating sensor inputs and outputs.

- Testing the embedded program.
- Verifying circuit connections.

Features

- Browser-based simulation environment.
- Supports ESP32 and Arduino boards.
- Real-time hardware simulation.
- Internet connectivity for IoT applications.

5.3 ThingsBoard

ThingsBoard is an open-source Internet of Things (IoT) platform used for collecting, storing, processing, and visualizing sensor data. The ESP32 sends telemetry data to the ThingsBoard cloud using the MQTT protocol. The dashboard displays real-time values of temperature, humidity, soil moisture, and pump status using gauges, charts, and indicators.

Purpose

- Cloud-based data monitoring.
- Real-time visualization of sensor data.
- Storage of telemetry data.
- Remote monitoring of the agriculture system.

Features

- Real-time dashboard.
- Device management.
- Telemetry storage.
- Alarm and notification support.
- MQTT, HTTP, and CoAP communication support.

5.4 MQTT Protocol

Message Queuing Telemetry Transport (MQTT) is a lightweight messaging protocol specifically designed for IoT applications. It follows a publish-subscribe communication model, allowing efficient data exchange between the ESP32 device and the ThingsBoard server. Due to its low bandwidth requirement and reliable communication, MQTT is widely used in IoT systems.

Purpose

- Sending sensor telemetry to the cloud.
- Reliable communication between ESP32 and ThingsBoard.

- Supporting real-time monitoring.

Features

- Lightweight communication protocol.
- Publish-subscribe architecture.
- Low network bandwidth usage.
- Fast and reliable data transfer.

5.5 Wi-Fi Communication

The **ESP32 DevKit V1** has an integrated Wi-Fi module that enables wireless communication with the internet. In this project, the ESP32 connects to the **Wokwi-GUEST** network during simulation and transmits sensor data to the ThingsBoard cloud platform using the MQTT protocol. Wi-Fi communication eliminates the need for wired networking and enables remote monitoring from any location.

Purpose

- Connect the ESP32 to the internet.
- Enable cloud communication.
- Transfer real-time sensor data.

Features

- Built-in IEEE 802.11 b/g/n support.
- Wireless internet connectivity.
- Fast and reliable communication.
- Supports IoT cloud platforms.

6. System Architecture

6.1 System Block Diagram

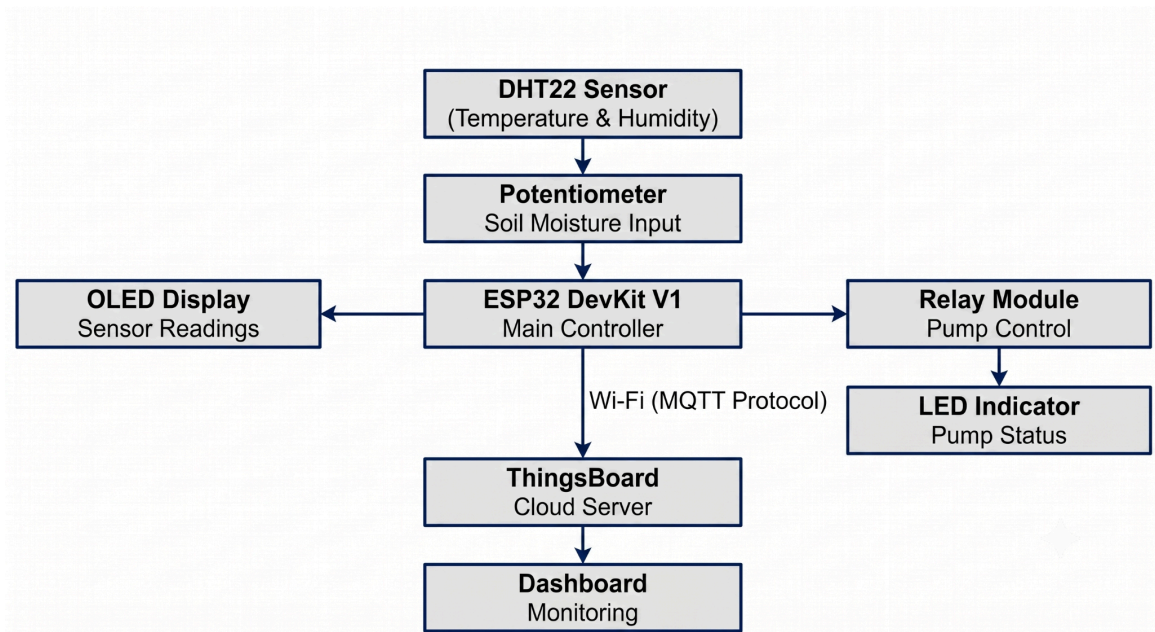


Diagram 6.1.1:Smart Agriculture System Block Diagram

6.2 Data Flow Diagram

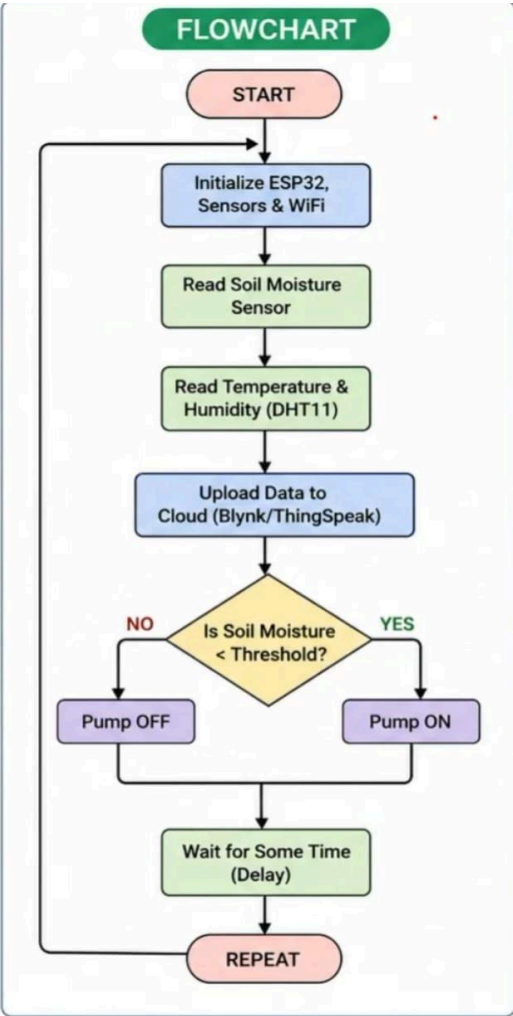


Diagram 6.2.1:Smart Agriculture System Data Flow Diagram

7. Circuit Design

7.1 Circuit Diagram

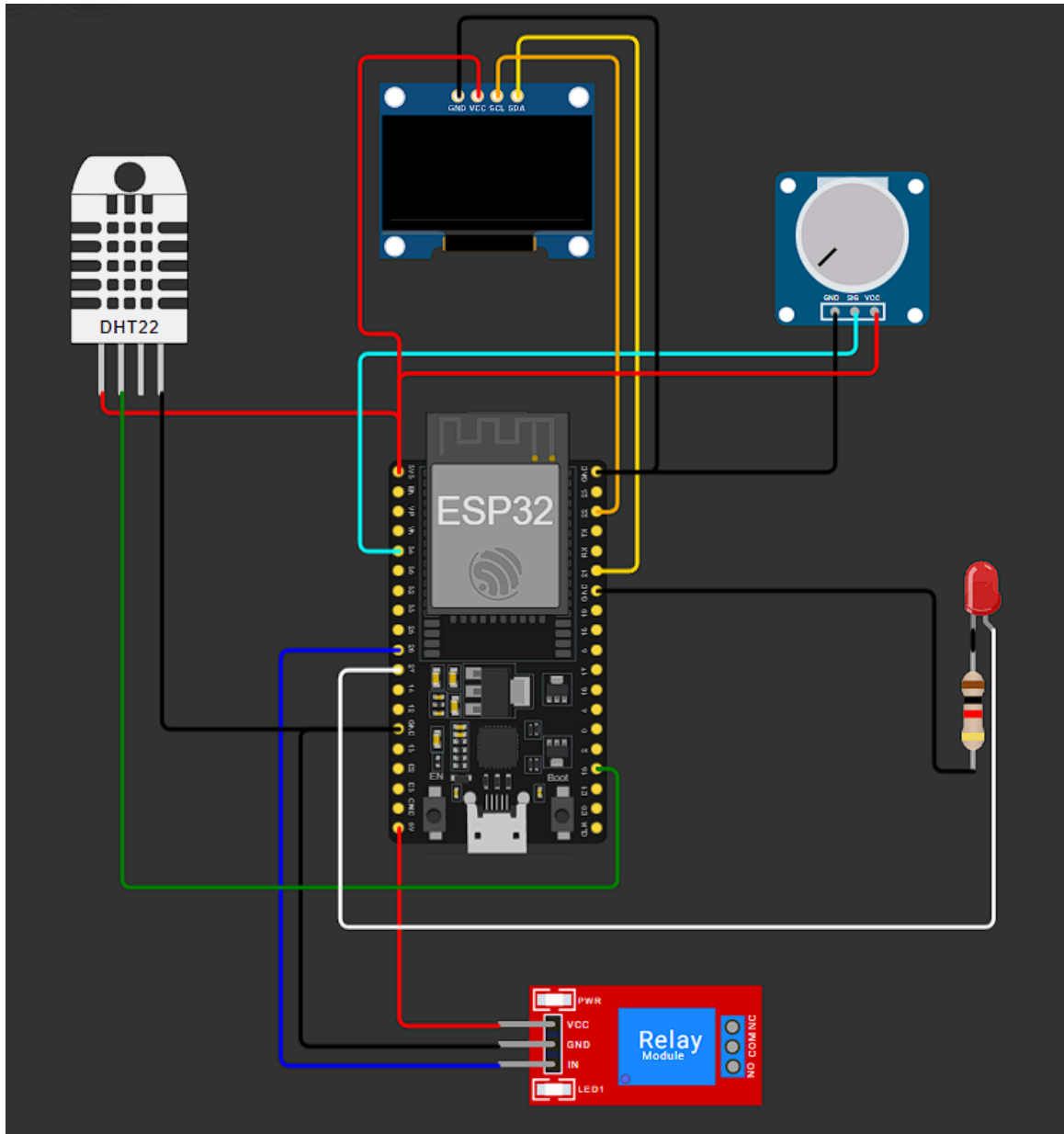


Diagram 7.1.1: Smart Agriculture system

7.2 Pin Configuration

Component	ESP32 Pin
DHT22 Data	GPIO 15
Potentiometer (SIG)	GPIO 34
Relay Module (IN)	GPIO 26
LED (Anode)	GPIO 27
OLED SDA	GPIO 21
OLED SCL	GPIO 22
OLED VCC	3.3V
OLED GND	GND
DHT22 VCC	3.3V
DHT22 GND	GND
Potentiometer VCC	3.3V
Potentiometer GND	GND
Relay VCC	5V (VIN)
Relay GND	GND

Table 7.2.1: Pin Configuration

8.Implementation Programming

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

//===== WiFi =====
const char* ssid = "Wokwi-GUEST";
const char* password = "";

//===== ThingsBoard =====
const char* mqtt_server = "thingsboard.cloud";
const char* accessToken = "f8dh1reSSlPvR92wSeeV";

WiFiClient espClient;
PubSubClient client(espClient);

//===== OLED =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

//===== DHT22 =====
#define DHTPIN 15
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

//===== Pins =====
#define SOIL_PIN 34
#define RELAY_PIN 26
#define LED_PIN 27
```

```

float temperature = 0;
float humidity = 0;
int soilRaw = 0;
int soilPercent = 0;
bool pumpStatus = false;

//-----WiFi+MQTT-----

void connectWiFi() {

    WiFi.begin(ssid, password);

    Serial.print("Connecting WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi Connected");
    Serial.print("IP : ");
    Serial.println(WiFi.localIP());
}

void reconnectMQTT() {

    while (!client.connected()) {

        Serial.print("Connecting to ThingsBoard... ");

        if (client.connect("ESP32Client", accessToken, NULL)) {
            Serial.println("Connected");
        } else {
            Serial.print("Failed, rc=");
            Serial.println(client.state());
            delay(3000);
        }
    }
}

```

```

void sendTelemetry() {

    String payload = "{";
    payload += "\"temperature\":";
    payload += String(temperature);
    payload += ",";
    payload += "\"humidity\":";
    payload += String(humidity);
    payload += ",";
    payload += "\"soilMoisture\":";
    payload += String(soilPercent);
    payload += ",";
    payload += "\"pump\":";
    payload += pumpStatus ? "true" : "false";
    payload += "}";

    client.publish("v1/devices/me/telemetry",
                  payload.c_str());

    Serial.println(payload);

}

void setup() {

    Serial.begin(115200);

    pinMode(RELAY_PIN, OUTPUT);
    pinMode(LED_PIN, OUTPUT);

    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(LED_PIN, LOW);

    dht.begin();

    Wire.begin(21,22);

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("OLED Failed");
        while (true);
    }

    display.clearDisplay();

```

```

display.display();

connectWiFi();

client.setServer(mqtt_server, 1883);
client.setBufferSize(512);
}

void loop() {

  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  if (!client.connected()) {
    reconnectMQTT();
  }

  client.loop();

  // Read Sensors
  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  soilRaw = analogRead(SOIL_PIN);

  // Convert ADC to %
  soilPercent = map(soilRaw, 0, 4095, 100, 0);

  // Automatic Pump Control
  if (soilPercent < 30) {

    digitalWrite(RELAY_PIN, HIGH);
    digitalWrite(LED_PIN, HIGH);
    pumpStatus = true;

  } else {

    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(LED_PIN, LOW);
    pumpStatus = false;
  }
}

```

```

}

// OLED Display
display.clearDisplay();

display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);

display.setCursor(0,0);
display.println("SMART AGRICULTURE");

display.setCursor(0,15);
display.print("Temp : ");
display.print(temperature);
display.println(" C");

display.setCursor(0,28);
display.print("Hum  : ");
display.print(humidity);
display.println(" %");

display.setCursor(0,41);
display.print("Soil : ");
display.print(soilPercent);
display.println(" %");

display.setCursor(0,54);

if (pumpStatus)
    display.print("Pump : ON");
else
    display.print("Pump : OFF");

display.display();

// Serial Monitor
Serial.println("-----");
Serial.print("Temperature : ");
Serial.println(temperature);

Serial.print("Humidity : ");
Serial.println(humidity);

```

```
Serial.print("Soil Moisture : ");  
Serial.println(soilPercent);  
  
Serial.print("Pump : ");  
Serial.println(pumpStatus ? "ON" : "OFF");  
  
// Send data to ThingsBoard  
sendTelemetry();  
  
delay(2000);  
}
```

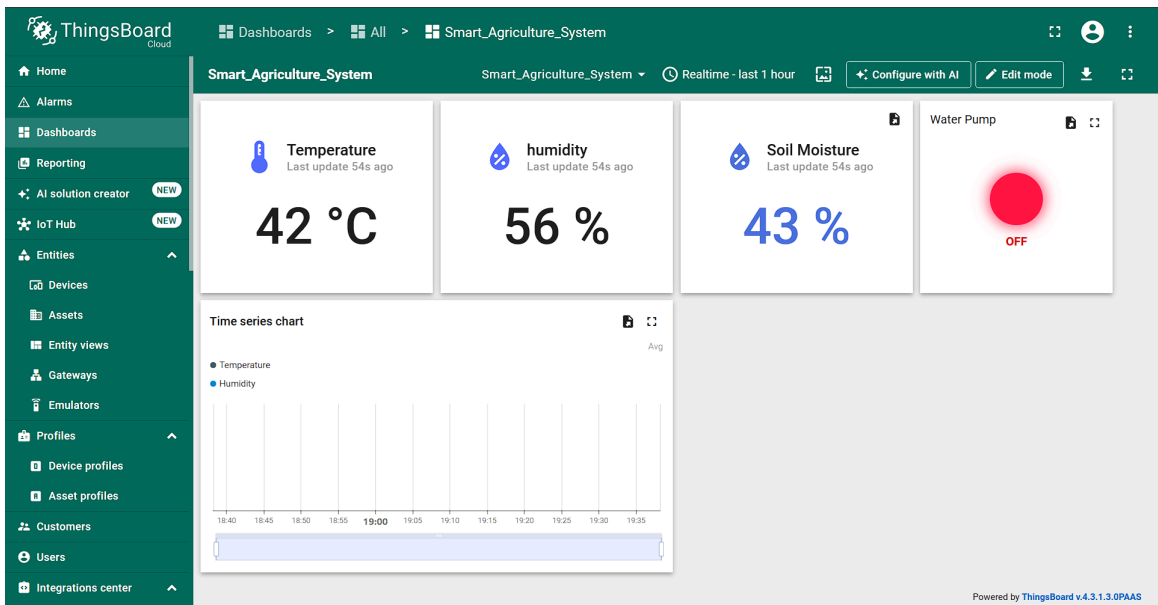
9. Testing and Results

9.1 Serial Monitor Reading

```
no 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Connecting WiFi.....
WiFi Connected
IP : 10.10.0.2
Connecting to ThingsBoard... Connected
-----
Temperature : 41.80
Humidity : 56.00
Soil Moisture : 43
Pump : OFF
{"temperature":41.80,"humidity":56.00,"soilMoisture":43,"pump":false}
-----
Temperature : 41.80
Humidity : 56.00
Soil Moisture : 43
Pump : OFF
{"temperature":41.80,"humidity":56.00,"soilMoisture":43,"pump":false}
-----
```

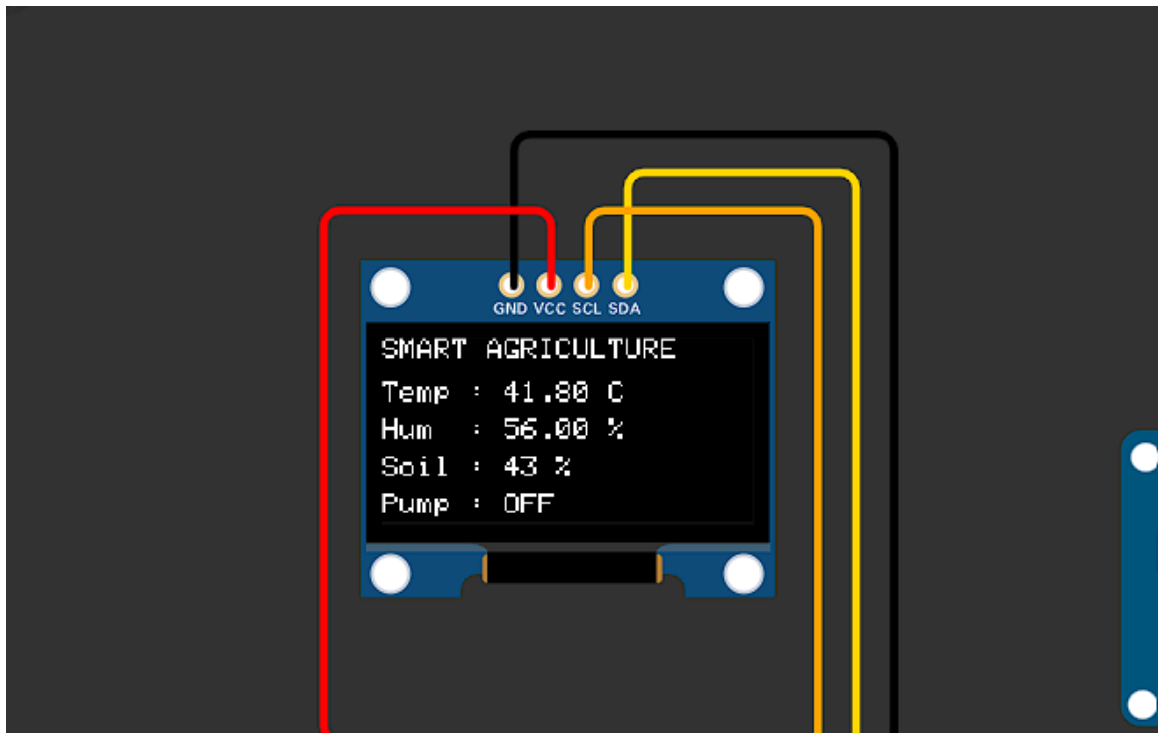
Screenshot 9.1.1:Serial Monitor Reading

9.2 Dashboard Reading



Screenshot 9.2.1:Dashboard Reading

9.3 OLED Output



Screenshot 9.3.1: OLED Output

10.Challenges Faced

During the development of the **IoT-Based Smart Agriculture Monitoring and Irrigation System**, several challenges were encountered and successfully resolved.

(1) MQTT Connection Issues

The ESP32 initially failed to connect to the ThingsBoard MQTT broker due to incorrect connection settings. The issue was resolved by verifying the MQTT server address, access token, and connection parameters.

(2) ThingsBoard Configuration

Configuring the ThingsBoard device and dashboard required correct device credentials and telemetry settings. After proper configuration, real-time sensor data was displayed successfully.

(3) Wokwi Simulation Setup

Some issues occurred during the Wokwi simulation, including incorrect pin connections and missing libraries. These were resolved by verifying the circuit and installing the required libraries.

(4) Sensor Calibration

The potentiometer used for soil moisture simulation required calibration to produce meaningful moisture percentage values. A suitable threshold was selected to enable automatic relay control.

11. Learning Outcomes

This project provided practical experience in developing an **IoT-Based Smart Agriculture Monitoring and Irrigation System**. During the implementation, I learned how to program the **ESP32 DevKit V1** using the Arduino IDE and interface various hardware components such as the DHT22 sensor, OLED display, relay module, and LED.

I gained knowledge of the **MQTT communication protocol** and understood how to transmit sensor data securely to the **ThingsBoard IoT platform** for real-time monitoring and visualization. I also learned how to design and test IoT circuits using the **Wokwi Simulator**, which helped in verifying the system before hardware implementation.

Through this project, I improved my understanding of **embedded system design, sensor interfacing, wireless communication, cloud integration, and automation**. I also developed problem-solving and debugging skills while resolving issues related to MQTT connectivity, cloud configuration, and sensor data processing.

Overall, this project enhanced my practical knowledge of IoT technologies and strengthened my ability to design and implement real-time smart monitoring systems for agricultural applications.

12. Conclusion

The **IoT-Based Smart Agriculture Monitoring and Irrigation System** was successfully designed and implemented using the **ESP32 DevKit V1**, **DHT22 sensor**, **OLED display**, **relay module**, and **ThingsBoard IoT platform**. The system effectively monitors temperature, humidity, and soil moisture levels in real time and automatically controls irrigation based on soil moisture conditions.

The integration of **Wi-Fi**, **MQTT communication**, and the **ThingsBoard cloud platform** enables remote monitoring and visualization of sensor data through an online dashboard. The OLED display provides local monitoring, while the relay module automates the irrigation process, reducing manual intervention and improving water management.

The project demonstrates the practical application of **Embedded Systems**, **Internet of Things (IoT)**, and **Cloud Computing** in modern agriculture. It provides a cost-effective and scalable solution that can be further enhanced by integrating additional sensors, weather forecasting, mobile applications, and AI-based decision-making for advanced smart farming applications.

Overall, the project successfully achieved its objectives and provided valuable practical experience in IoT system development, cloud integration, sensor interfacing, and automation technologies.

13. References

The following references were used during the design, development, and implementation of the **IoT-Based Smart Agriculture Monitoring and Irrigation System**.

1. Arduino. *Arduino IDE Documentation*. Available at: <https://www.arduino.cc/>
2. Espressif Systems. *ESP32 DevKit V1 Documentation*. Available at: <https://www.espressif.com/>
3. ThingsBoard. *ThingsBoard Documentation*. Available at: <https://thingsboard.io/docs/>
4. Wokwi. *Online ESP32 Simulator*. Available at: <https://wokwi.com/>
5. MQTT.org. *MQTT Version 3.1.1 Protocol Specification*. Available at: <https://mqtt.org/>
6. Adafruit Industries. *DHT22 Temperature and Humidity Sensor Guide*. Available at: <https://learn.adafruit.com/dht>
7. Adafruit Industries. *SSD1306 OLED Display Library Documentation*. Available at: <https://learn.adafruit.com/monochrome-oled-breakouts>
8. Arduino Library Manager. *PubSubClient MQTT Library*. Available at: <https://github.com/knolleary/pubsubclient>
9. Arduino Library Manager. *Adafruit GFX Library*. Available at: <https://github.com/adafruit/Adafruit-GFX-Library>
10. Arduino Library Manager. *Adafruit SSD1306 Library*. Available at: https://github.com/adafruit/Adafruit_SSD1306
11. OpenAI. *ChatGPT (GPT-5.5)*. Available at: <https://chatgpt.com/>